

- 验证中断向量地址
- 确认中断源是否正确配置

问题：系统中断处理后崩溃

- 检查栈指针保存和恢复
- 验证上下文保存的完整性
- 确认中断处理函数没有破坏调用约定

问题：中断频率异常

- 检查时钟设置参数
- 验证SBI调用是否正确
- 确认时间计算没有溢出

思考题

1. 中断设计：

- 为什么时钟中断需要在M模式处理后再委托给S模式？
- 如何设计一个支持中断优先级的系统？

2. 性能考虑：

- 中断处理的时间开销主要在哪里？如何优化？
- 高频率中断对系统性能有什么影响？

3. 可靠性：

- 如何确保中断处理函数的安全性？
- 中断处理中的错误应该如何处理？

4. 扩展性：

- 如何支持更多类型的中断源？
- 如何实现中断的动态路由？

5. 实时性：

- 当前实现的中断延迟特征如何？
 - 如何设计一个满足实时要求的中断系统？
-

实验5：进程管理与调度

实验目标

通过深入分析xv6的进程管理机制，理解操作系统如何创建、管理和调度进程，实现完整的进程生命周期管理和简单的调度算法。

核心学习资料

进程管理理论基础

- 操作系统概念 第3-5章：进程、线程、CPU调度
- xv6手册 第2-4章：操作系统组织、页表、陷阱和系统调用
- RISC-V调用约定： <https://riscv.org/wp-content/uploads/2015/01/riscv-calling.pdf>

xv6进程管理源码分析

- `kernel/proc.h` - 进程结构体定义
 - 重点： `struct proc` 的字段含义和生命周期
- `kernel/proc.c` - 进程管理核心函数
 - 重点函数： `allocproc()`, `fork()`, `exit()`, `wait()`, `scheduler()`
 - 学习要点：进程状态转换、内存管理、调度策略
- `kernel/swtch.S` - 上下文切换汇编代码
 - 理解：寄存器保存策略、栈切换机制
- `kernel/sysproc.c` - 进程相关系统调用
 - 重点： `sys_fork()`, `sys_exit()`, `sys_wait()`, `sys_kill()`

任务列表

任务1：深入理解进程抽象

学习重点：

1. 分析xv6的进程结构体：

```
1  struct proc {
2      struct spinlock lock;
3      enum procstate state;    // 进程状态
4      void *chan;              // 等待通道
5      int killed;              // 是否被杀死
6      int xstate;              // 退出状态
7      int pid;                 // 进程ID
8      pagetable_t pagetable;   // 用户页表
9      struct trapframe *trapframe; // 陷阱帧
10     struct context context;   // 调度上下文
11     // ...更多字段
12 };
```

- 每个字段的作用是什么？
 - 进程状态转换图是怎样的？
 - 为什么需要锁保护？
2. 理解进程生命周期：

- UNUSED → USED → RUNNABLE → RUNNING → SLEEPING → ZOMBIE
- 每个状态转换的触发条件是什么？
- 哪些操作需要原子性保护？

深入思考：

- 为什么需要ZOMBIE状态？
- 进程表的大小限制有什么影响？
- 如何防止进程ID重复？

任务2：分析xv6的进程创建机制

代码阅读指导：

1. 研读 `allocproc()` 函数：

- 如何在进程表中找到空闲槽位？
- 进程ID是如何分配的？
- 用户栈是如何设置的？
- 陷阱帧的初始化过程

2. 深入理解 `fork()` 实现：

```
1  int fork(void) {  
2      // 1. 分配新进程结构  
3      // 2. 复制用户内存  
4      // 3. 复制陷阱帧  
5      // 4. 设置返回值  
6      // 5. 标记为RUNNABLE  
7  }
```

- 为什么父子进程有不同的返回值？
- 内存复制是如何实现的？
- 失败时的资源清理策略

3. 分析进程退出机制：

- `exit()` 与 `wait()` 的协作关系
- 资源回收的时机和方式
- 孤儿进程的处理

关键问题：

- `fork()` 的性能瓶颈在哪里？
- 如何实现写时复制优化？

任务3：设计你的进程管理系统

设计要求：

1. 确定进程结构体设计

2. 选择合适的进程表组织方式
3. 设计进程ID分配策略

核心接口设计：

```
1 // 进程管理基本接口
2 struct proc* alloc_process(void);           // 分配进程结构
3 void free_process(struct proc *p);          // 释放进程资源
4 int create_process(void (*entry)(void));    // 创建新进程
5 void exit_process(int status);              // 终止当前进程
6 int wait_process(int *status);              // 等待子进程
7
8 // 你需要考虑的设计问题：
9 // 1. 进程表用数组还是链表？
10 // 2. 如何高效查找特定PID的进程？
11 // 3. 是否需要进程组和会话的概念？
12 // 4. 如何处理进程资源限制？
```

实现策略：

1. 先实现基本的进程创建和销毁
2. 再添加父子关系管理
3. 最后考虑性能优化

任务4：实现上下文切换机制

参考xv6的swtch.S，理解：

1. 上下文切换的本质：
 - 哪些寄存器需要保存？
 - 为什么不保存所有寄存器？
 - 调用者保存 vs 被调用者保存的区别
2. 栈的切换：
 - 内核栈 vs 用户栈的管理
 - 栈指针的保存和恢复
 - 栈溢出的检测和预防

实现挑战：

```
1 // 上下文结构体设计
2 struct context {
3     uint64 ra; // 返回地址
4     uint64 sp; // 栈指针
5     // 需要保存哪些其他寄存器？
6     // 为什么这样选择？
7 };
8
9 // 上下文切换函数
10 void swtch(struct context *old, struct context *new);
```

关键技术点:

- 上下文切换必须是原子操作
- 中断状态的管理
- 多级栈的处理

任务5: 实现调度器

参考xv6的调度策略:

1. 分析 `scheduler()` 函数:

- 轮转调度的实现方式
- 如何避免忙等待?
- 为什么需要开启中断?

2. 理解调度时机:

- 主动调度 vs 抢占调度
- `yield()` 函数的作用
- 时钟中断如何触发调度

调度器设计考虑:

```
1 void scheduler(void) {
2     struct proc *p;
3     struct cpu *c = mycpu();
4
5     c->proc = 0;
6     for(;;) {
7         // 开启中断, 允许设备中断
8         intr_on();
9
10        // 你的调度算法:
11        // 1. 如何选择下一个运行的进程?
12        // 2. 如何处理优先级?
13        // 3. 如何避免饥饿?
14        // 4. 如何平衡公平性和效率?
15
16        for(p = proc; p < &proc[NPROC]; p++) {
17            acquire(&p->lock);
18            if(p->state == RUNNABLE) {
19                // 找到可运行进程, 切换过去
20                p->state = RUNNING;
21                c->proc = p;
22                swtch(&c->context, &p->context);
23                c->proc = 0;
24            }
25            release(&p->lock);
26        }
27    }
```

扩展调度算法:

- 优先级调度
- 多级反馈队列
- 完全公平调度器 (CFS)

任务6: 实现进程同步原语**基于xv6的sleep/wakeup机制:**

1. 理解条件变量的概念:

```

1 // 等待条件满足
2 void sleep(void *chan, struct spinlock *lk);
3 // 唤醒等待特定条件的进程
4 void wakeup(void *chan);

```

2. 分析典型使用模式:

- a. 生产者-消费者问题
- b. 读者-写者问题
- c. 信号量的实现

实现要点:

- 避免lost wakeup问题
- 锁的正确使用
- 中断状态的管理

测试与调试策略**进程创建测试**

```

1 void test_process_creation(void) {
2     printf("Testing process creation...\n");
3
4     // 测试基本的进程创建
5     int pid = create_process(simple_task);
6     assert(pid > 0);
7
8     // 测试进程表限制
9     int pids[NPROC];
10    int count = 0;
11    for (int i = 0; i < NPROC + 5; i++) {
12        int pid = create_process(simple_task);
13        if (pid > 0) {
14            pids[count++] = pid;
15        } else {
16            break;

```

```

17     }
18 }
19 printf("Created %d processes\n", count);
20
21 // 清理测试进程
22 for (int i = 0; i < count; i++) {
23     wait_process(NULL);
24 }
25 }

```

调度器测试

```

1 void test_scheduler(void) {
2     printf("Testing scheduler...\n");
3
4     // 创建多个计算密集型进程
5     for (int i = 0; i < 3; i++) {
6         create_process(cpu_intensive_task);
7     }
8
9     // 观察调度行为
10    uint64 start_time = get_time();
11    sleep(1000); // 等待1秒
12    uint64 end_time = get_time();
13
14    printf("Scheduler test completed in %lu cycles\n",
15           end_time - start_time);
16 }

```

同步机制测试

```

1 void test_synchronization(void) {
2     // 测试生产者-消费者场景
3     shared_buffer_init();
4
5     create_process(producer_task);
6     create_process(consumer_task);
7
8     // 等待完成
9     wait_process(NULL);
10    wait_process(NULL);
11
12    printf("Synchronization test completed\n");
13 }

```

调试建议

进程状态调试

```
1 void debug_proc_table(void) {  
2     printf("=== Process Table ===\n");  
3     for (int i = 0; i < NPROC; i++) {  
4         struct proc *p = &proc[i];  
5         if (p->state != UNUSED) {  
6             printf("PID:%d State:%d Name:%s\n",  
7                 p->pid, p->state, p->name);  
8         }  
9     }  
10 }
```

调度器调试

- 在调度器中添加统计信息
- 跟踪进程运行时间
- 分析调度延迟

内存泄漏检测

- 跟踪进程创建和销毁
- 检查页表释放
- 监控进程表使用情况

思考题

1. 进程模型：
 - 为什么选择这种进程结构设计？
 - 如何支持轻量级线程？
 2. 调度策略：
 - 轮转调度的公平性如何？
 - 如何实现实时调度？
 3. 性能优化：
 - `fork()` 的性能瓶颈如何解决？
 - 上下文切换开销如何降低？
 4. 资源管理：
 - 如何实现进程资源限制？
 - 如何处理进程资源泄漏？
 5. 扩展性：
 - 如何支持多核调度？
 - 如何实现负载均衡？
-